SOFTWARE ENGINEERING · CHAPTER 07

Lab-07: Introduction to UML and Use Case Diagrams

Model one system, the ITC Campus Library System, in five steps: find the actors and the boundary, draw a first use case diagram, refine it with «include», «extend» and generalization, write fully dressed use case specifications with Gherkin scenarios, and close the loop with a traceability matrix and a diagram review. The challenge adds an inter-library-loan subsystem, a package diagram, a scope document and a test plan skeleton for Chapter 06 style tests.

UML 2.5.1 PlantUML (text) + PNG export JDK 25 to run plantuml.jar ≈ 5 hours 5 tasks + 1 challenge

How to work through this lab. The tasks build on each other inside *one* modelling case study. Every diagram is submitted as **source text** (`.puml`, or Mermaid/draw.io XML if you prefer another tool) **and** a PNG or SVG export. Commit after every task (`git tag task-N`). Tick the acceptance checklist as you go (it is saved in your browser). Open hints only after 10 minutes of being stuck.

0 Setup and case study — 20 min

TOOLING

1. Verify Java (PlantUML is a Java program) and download PlantUML:

```
java -version        # 25
mkdir -p lab-07/tools lab-07/diagrams lab-07/specs
curl -L -o lab-07/tools/plantuml.jar \
  https://github.com/plantuml/plantuml/releases/latest/download/plantuml.jar
# smoke test: renders lab-07/diagrams/hello.png without Graphviz
java -jar lab-07/tools/plantuml.jar -Playout=smetana -tpng lab-
07/diagrams/hello.puml
```

Alternatives: the *PlantUML* extension in VS Code or IntelliJ (preview on save), plantuml.com online, draw.io (export XML + PNG). Whatever you use, the `.puml` text is the graded artefact.

2. Create `lab-07/diagrams/hello.puml` and make sure it renders:

```
@startuml hello
left to right direction
actor Member
```

```
rectangle "Campus Library System" {
  usecase "Search catalogue" as UC5
}
Member -- UC5
@enduml
```

3. Initialise git in `lab-07/`, add a `README.md` with your name and the tool you chose, and commit.

CASE STUDY: ITC CAMPUS LIBRARY SYSTEM

The ITC Campus Library serves about 6 000 students and 400 staff in two buildings and still runs on a spreadsheet and a paper card file. The university has approved a new **Campus Library System**. Members identify themselves with their university card; authentication is done by the university's identity provider (IdP, OpenID Connect). Fines are paid through the university's payment provider **CampusPay**, and all e-mail goes through the university mail relay. The library director wants self-service borrowing and returning at desk terminals and kiosks, reservations, automatic fines, a catalogue that librarians maintain themselves, and fewer phone calls about "is my book back yet".

Raw requirements as collected in the interviews (deliberately unpolished):

Id	Requirement
R1	Students and staff borrow books by presenting their university card at a desk terminal or a self-service kiosk.
R2	Members return books at the desk or the kiosk; the loan is closed immediately.
R3	A member can reserve a book that is currently on loan and is notified when it becomes available; a reserved book is held for 3 days.
R4	Late returns incur a fine of 0.50 per day; a member with an outstanding fine above 10.00 cannot borrow.
R5	Fines are paid online through CampusPay, the university's external payment provider.
R6	Librarians manage the catalogue: add titles and copies, update details, withdraw damaged copies.
R7	A librarian can extend a loan or waive a fine, recording a reason.
R8	The system sends e-mail notifications (due-date reminder 2 days before, reservation ready, fine issued) via the university mail relay.
R9	Members can search the catalogue and see their current loans, reservations and fines; anonymous visitors can only search.
R10	Every action that needs a member identity authenticates the university card through the university identity provider; the library never stores passwords.

REPOSITORY LAYOUT

```
lab-07/
├── README.md                ← name, tool, how to render
├── tools/plantuml.jar
├── actors.md                ← Task 1
├── diagrams/
│   ├── context.puml / .png  ← Task 1
│   ├── library-v1.puml / .png← Task 2
│   ├── library-v2.puml / .png← Task 3
│   └── library-final.puml   ← Task 5 (+ .png)
├── specs/
│   ├── UC-01-borrow-book.md ← Task 4 (three files)
│   └── UC-01-borrow-book.feature
├── traceability.md          ← Task 5
└── review.md                ← Task 5
```

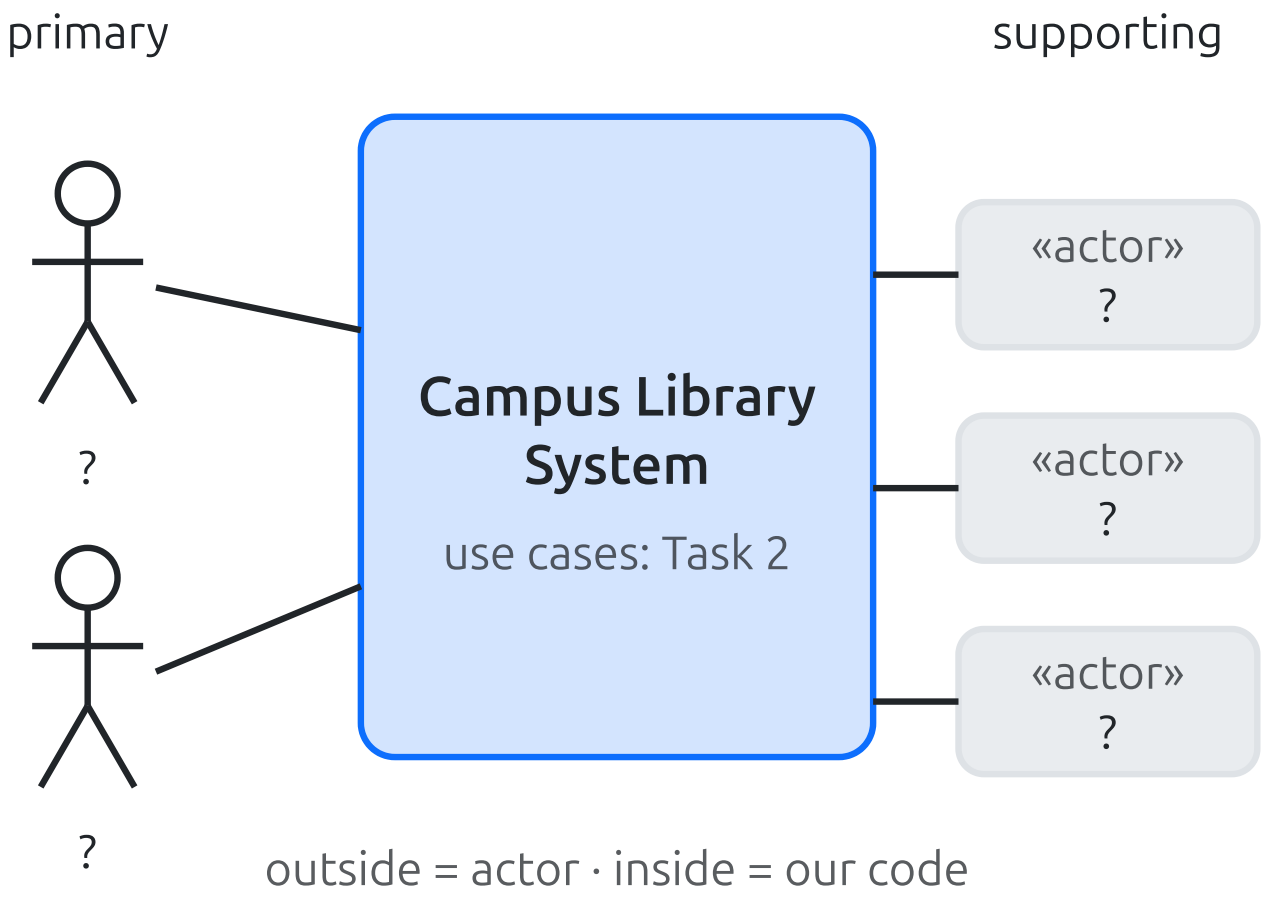
1 Actors and system boundary Easy 30 min

GOAL

Decide who and what is outside the Campus Library System, classify each actor as primary or supporting, and write down where the boundary runs and what is explicitly out of scope.

STEPS

1. Read R1-R10 and underline every noun that could be a person, an organisation or an external system. Put each into one of three columns in `actors.md`: *actor*, *inside the system*, *not an actor at all* (with a one-line reason).
2. Fill the actor table: name (a role, singular), kind (person / system), **primary or supporting**, the goals it has (for primary) or the service it provides (for supporting), and the requirements it comes from.
3. Decide the boundary and name it exactly `Campus Library System`. List what is inside (loans, reservations, fines, catalogue, notifications logic).
4. Write an **out-of-scope list** with at least four items, each with a reason (for example: managing IdP accounts, refunds inside CampusPay, purchasing new titles, delivering the e-mail).
5. Draw `diagrams/context.puml`: all actors and the boundary rectangle with **no use cases yet**, primary actors on the left, supporting on the right; render it to `context.png`.
6. Decide and justify in `actors.md`: is the *self-service kiosk* an actor? Is *Time* (the due-date reminder trigger) an actor? Is the *anonymous visitor* an actor?



STARTER: `ACTORS.MD`

```
# Actors and boundary - Campus Library System

## Candidates
| Noun from R1-R10 | Actor / inside / not an actor | Reason |
|------------------|-------------------------------|--------|
| Student          | actor                         | TODO   |
| Book             | inside                        | TODO   |
| Kiosk            | TODO                          | TODO   |

## Actors
| Id | Actor | Kind (person/system) | Primary / supporting | Goals or service |
From |
|----|-------|----------------------|----------------------|------------------|-
-----|
| A1 | Member | person | primary | borrow, return, reserve, pay fine, ... | R1-
R5, R9 |
| A2 | TODO   |        |         |                                        |
|

## Boundary: "Campus Library System"
Inside : TODO
```

```
Out of scope (with reason):
1. TODO
2. TODO
```

EXPECTED OUTPUT (EXCERPT OF THE ACTOR TABLE)

```
| A1 | Member (Student, Staff) | person | primary    | borrow, return, reserve,
pay fine, view account | R1-R5, R9 |
| A2 | Librarian               | person | primary    | manage catalogue, extend
loan, waive fine       | R6, R7    |
| A3 | Visitor (anonymous)     | person | primary    | search catalogue
| R9        |
| A4 | University IdP          | system | supporting | authenticates the
university card              | R1, R10   |
| A5 | CampusPay               | system | supporting | collects fine payments
| R5        |
| A6 | Mail relay              | system | supporting | delivers notification e-
mails                   | R8        |
Rejected: Book, Loan, Fine, Reservation (inside); Kiosk (a channel / UI, not a
role);
          Database (inside); "Library director" (a stakeholder, never interacts)
```

Show hints

- Test for an actor: "Can this be replaced by a different person or system without changing our code's purpose?" A kiosk and a desk terminal are two *ways* the same Member reaches the system, so they are channels, not actors.
- Students and staff have the same goals (R1-R5, R9); model them as one *Member* now. Task 3 asks whether they deserve a generalization (hint: R4 and loan periods).
- Supporting actors are what the system *calls*: the IdP, CampusPay, the mail relay. The director wants reports but never interacts: an offstage stakeholder, listed in the specs later.
- Time-triggered behaviour (R8 reminders) is usually shown either with a «time» actor or as a note "triggered daily at 06:00". Pick one and be consistent.
- A PlantUML external system: `actor "University IdP" as IdP <<system>>`. Declare primary actors before the rectangle and supporting ones after it so they land on the correct side with `left to right direction`.

ACCEPTANCE CHECKLIST

- [ ] `actors.md` lists at least five actors, each marked person/system and primary/supporting, with the requirements it comes from.
- [ ] Book, Loan, Fine, Database and Kiosk are explicitly rejected as actors with a reason.

- [ ] The out-of-scope list has at least four items with reasons, and the boundary is named `Campus Library System`.
- [ ] `context.puml` renders with primary actors left, supporting actors right, and no use case inside the rectangle yet.

2 First use case diagram (associations only) Easy 45 min

GOAL

Turn R1-R10 into 8 to 12 user-goal use cases with stable ids, connect them to the actors from Task 1 with plain associations, and produce a diagram that reads in ten seconds.

STEPS

1. For every requirement write the actor's goal as *verb + object*. Merge duplicates and drop anything that is a step rather than a goal (scanning a card, calculating a fine, sending one e-mail). Aim for 8-12 use cases.
2. Give each use case a stable id `UC-01` ... in a table in `README.md`: id, name, primary actor, goal in one sentence, requirements covered.
3. Create `diagrams/library-v1.puml` from the starter: boundary `Campus Library System`, actors from Task 1, use cases with ids as aliases, **associations only** (no include, extend or generalization yet).
4. Connect supporting actors to the use cases that need them (IdP, CampusPay, mail relay). Anonymous visitors get exactly one association.
5. Render with `-Playout=smetana -tpng`, then check the clarity rules: primary actors left, supporting right, no crossing lines you could avoid, every ellipse has a verb, 5-12 ellipses.
6. Read the diagram aloud as sentences ("A Member borrows a book, authenticated by the University IdP") and fix any sentence that sounds wrong. Commit and tag `task-2`.

STARTER: `DIAGRAMS/LIBRARY-V1.PUML`

```
@startuml library-v1
left to right direction
skinparam packageStyle rectangle
' ---- primary actors (left) ----
actor Member
actor Librarian
actor Visitor
rectangle "Campus Library System" {
  usecase "Borrow book"      as UC01
```

```
  usecase "Return book"      as UC02
  usecase "Reserve book"     as UC03
  ' TODO: UC04 ... UC12 (goals from R4-R9), verb + object
}
' ---- supporting actors (right) ----
actor "University IdP" as IdP <<system>>
' TODO: CampusPay, Mail relay
Member -- UC01
Member -- UC02
Member -- UC03
UC01 -- IdP
' TODO: remaining associations
@enduml
```

EXPECTED OUTPUT (WHAT THE RENDERED DIAGRAM MUST SHOW)

```
Boundary "Campus Library System" with 9-11 ellipses, for example:
  UC-01 Borrow book        Member     (R1, R4, R10)   -- IdP
  UC-02 Return book        Member     (R2)
  UC-03 Reserve book       Member     (R3)            -- Mail relay
  UC-04 Pay fine           Member     (R4, R5)        -- CampusPay
  UC-05 Search catalogue   Member, Visitor (R9)
  UC-06 View my account    Member     (R9)            -- IdP
  UC-07 Manage catalogue   Librarian  (R6)
  UC-08 Extend loan        Librarian  (R7)
  UC-09 Waive fine         Librarian  (R7)
  UC-10 Send due-date reminders   «time» / system-triggered (R8) -- Mail relay
Left column: Member, Librarian, Visitor.  Right column: IdP, CampusPay, Mail
relay.
No dashed arrows, no triangles: associations only.
$ java -jar tools/plantuml.jar -checkonly diagrams/library-v1.puml   → no output
= OK
```

Show hints

- "Can the actor go for lunch happy after this?" is the sea-level test. *Scan card* fails it; *Borrow book* passes.
- R8 hides three notifications. They are not three goals of a Member: model one time-triggered use case for the reminder and treat "reservation ready" and "fine issued" as steps inside UC-03 and UC-02 (the mail relay then connects to those).
- R4 contains a business rule (fine per day, borrowing block) and a goal (pay the fine). Only the goal becomes an ellipse; the rule goes into the specification in Task 4.
- Crossing lines: reorder the `usecase` declarations so that use cases of the same actor are adjacent. `Member -[hidden]down- Librarian` pins the actor order.
- Keep the alias equal to the id (`as UC01`): the traceability matrix in Task 5 will use it.

ACCEPTANCE CHECKLIST

- [ ] 8 to 12 use cases, each named verb + object, each with an id and one primary actor in the README table.
- [ ] Every requirement R1-R10 is covered by at least one use case, and no use case is a UI step or an internal computation.
- [ ] Only associations are used; supporting actors are connected to the use cases that call them and sit on the right.
- [ ] `library-v1.puml` passes `-checkonly` and `library-v1.png` is committed with it.

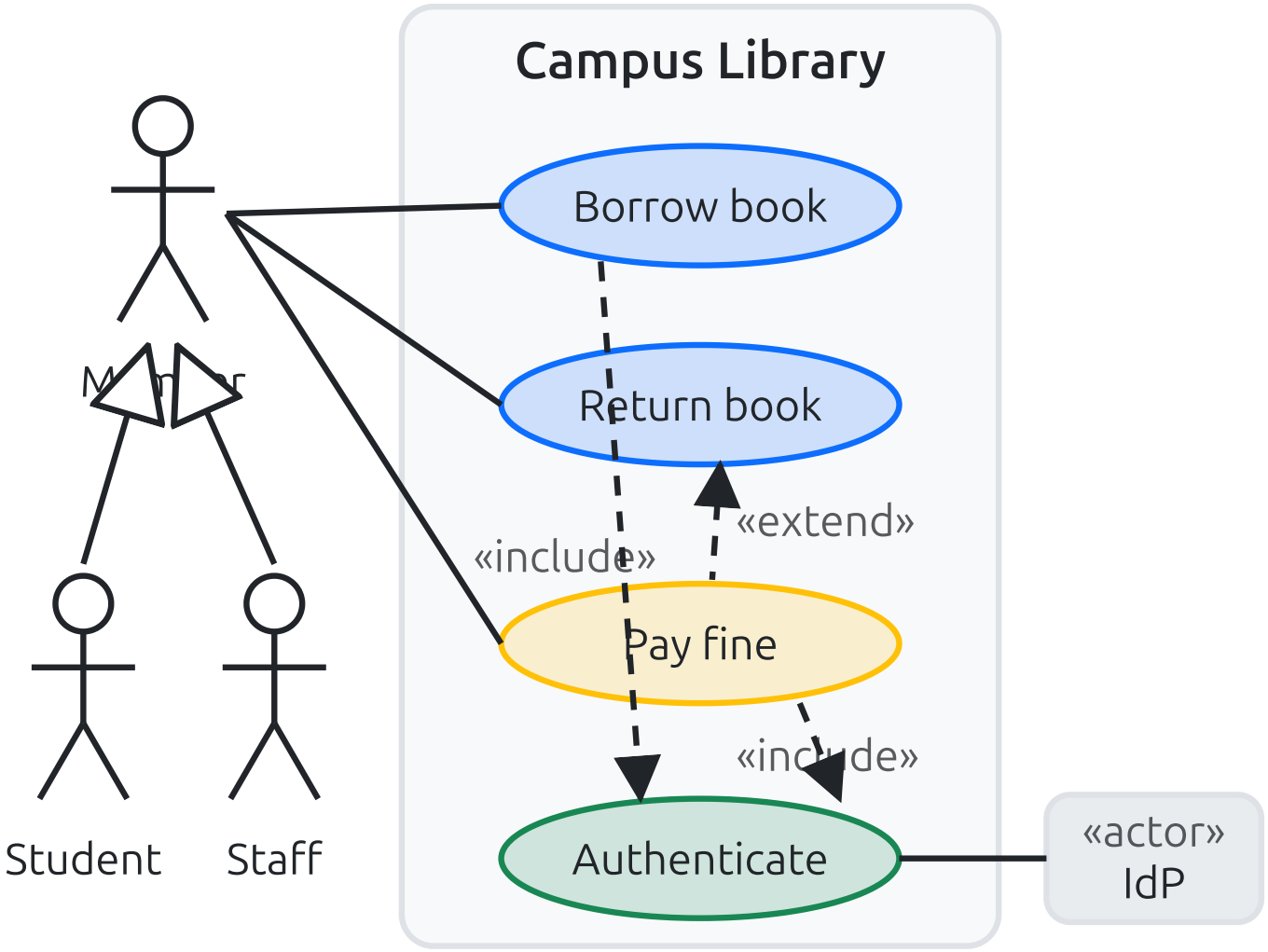
3 Include, extend and generalization **Medium** **60 min**

GOAL

Refactor `library-v1` into `library-v2`: extract the shared subfunction *Authenticate* with «include», attach *Pay fine* to *Return book* with «extend», introduce actor generalization, and justify every relationship in a table (including the ones you decided *not* to draw).

STEPS

1. Copy `library-v1.puml` to `library-v2.puml`. Add `usecase "Authenticate" as UC11` and draw `UCxx ..> UC11 : <<include>>` from every use case that needs a member identity (R10). Move the IdP association from those use cases to *Authenticate* only.
2. Model R4: a member returning a late book may pay the fine right away. Add an extension point `fine outstanding` to *Return book* (a note) and draw `UC04 ..> UC02 : <<extend>>`. *Pay fine* must remain reachable directly by the Member too.
3. Introduce actor generalization: `actor Student` and `actor Staff` with `Student --|> Member`, `Staff --|> Member`. Decide which associations stay on *Member* and which belong to a child only (staff can, say, *Request purchase*, a new UC-12 from a director interview: "staff may suggest titles to buy").
4. Consider use case generalization for R5: *Pay fine* by CampusPay online versus paying cash at the desk (recorded by a Librarian). Draw it only if both children share the goal and most steps; otherwise explain why an extension or a scenario is enough.
5. Write the justification table in `README.md` (columns: relationship, from → to, reason, alternative considered). Include at least one relationship you rejected.
6. Re-check arrow directions and render; count: associations must still outnumber includes and extends. Tag `task-3`.



STARTER: FRAGMENT FOR `DIAGRAMS/LIBRARY-V2.PUML`

```
' ---- actor generalization ----
actor Member
actor Student
actor Staff
Student --|> Member
Staff   --|> Member

rectangle "Campus Library System" {
  usecase "Borrow book"  as UC01
  usecase "Return book"  as UC02
  usecase "Pay fine"     as UC04
  usecase "Authenticate" as UC11
  note right of UC02
    extension points
    fine outstanding
  end note
}
actor "University IdP" as IdP <<system>>
```

```
' ---- include: base ..> included ----
UC01 ..> UC11 : <<include>>
UC02 ..> UC11 : <<include>>
' TODO: every other use case that needs an identity (R10)

' ---- extend: extension ..> base, with condition ----
UC04 ..> UC02 : <<extend>>\n[fine outstanding]

UC11 -- IdP
' TODO: Member / Student / Staff associations; CampusPay; Mail relay
```

EXPECTED OUTPUT (JUSTIFICATION TABLE, EXCERPT)

```
| Relationship   | From -> To                    | Reason
| Alternative considered            |
|---------------|-------------------------------|-----------------------------
----------------------|-----------------------------------|
| include        | UC01, UC02, UC03, UC04, UC06 -> UC11 | R10: identity needed
every time, same steps | copy the steps into 5 specs (dup) |
| extend         | UC04 -> UC02 [fine outstanding] | optional: only late
returns; UC02 unaware of fines | include (wrong: not every return) |
| generalization | Student, Staff --|> Member    | same goals; Staff adds UC12
Request purchase        | two unrelated actors (13 lines)   |
| REJECTED gen.  | Pay online / Pay at desk --|> Pay fine | steps differ only in
step 4: one scenario each | kept as extensions 4a/4b in spec  |
Counts in library-v2: associations 14 · include 5 · extend 1 · generalization 2
```

Show hints

- After extracting *Authenticate*, the IdP should have exactly one association left. If a use case still talks to the IdP directly, ask why.
- «extend» is right for *Pay fine* from *Return book* because a return works without it; «include» would say every return pays a fine. Write the condition on the arrow label (`\n` gives a second line in PlantUML).
- Generalization pays off only if the child adds or removes associations. If Student and Staff end up with identical lines, keep a single *Member* and note the loan-period difference in the specification.
- *Search catalogue* must not include *Authenticate* (R9: visitors search anonymously). A common mistake is to include it everywhere.
- PlantUML draws the hollow triangle for `--|>`; check in the PNG that the triangle sits at *Member*, not at the child.

ACCEPTANCE CHECKLIST

- [ ] *Authenticate* is included by every identity-requiring use case, is the only use case associated with the IdP, and is not included by *Search catalogue*.

- [ ] *Pay fine* extends *Return book* with a condition and a named extension point, and stays directly reachable by the Member.
- [ ] Student and Staff specialise Member with the triangle at Member; at least one association exists only on a child.
- [ ] The justification table covers every include, extend and generalization plus one rejected relationship; all arrows point the right way in the PNG.

4 Use case specifications (Cockburn style) and Gherkin

Medium

75 min

GOAL

Write three fully dressed specifications, **UC-01 Borrow book**, **UC-02 Return book** and **UC-04 Pay fine**, with preconditions, guarantees, a numbered main success scenario and at least two extensions each; then convert UC-01 into Gherkin scenarios that Chapter 06 tests could implement.

STEPS

1. Create `specs/UC-01-borrow-book.md` from the starter template. Fill the header first: scope, level, primary actor, goal in one sentence, stakeholders and their interests (include the library director as offstage), trigger.
2. Write **preconditions** that are true before the trigger (identity, card status) and two **postconditions**: the minimal guarantee (what holds even on failure) and the success guarantee.
3. Write the main success scenario as 6-10 numbered steps alternating "Member ..." and "System ..."; include the «include» of *Authenticate* as a single step ("System authenticates the Member (UC-11)"). No UI words (button, screen, click).
4. Add at least two extensions with step numbers (for UC-01 at least: outstanding fine above 10.00 from R4, copy already reserved by someone else from R3). State where each rejoins or that the use case fails.
5. Repeat for `UC-02-return-book.md` (extension: late return creates a fine and offers UC-04; another: copy reported damaged) and `UC-04-pay-fine.md` (extensions: CampusPay declines; CampusPay unreachable). Reference the business rules of R3 and R4 with their numbers.
6. Convert UC-01 to `specs/UC-01-borrow-book.feature`: one *Background* for the preconditions, one scenario for the main success scenario, one per extension. Keep the

extension ids in the scenario titles.

7. Cross-check: every step of the scenario appears in exactly one Gherkin line; every guarantee is a `Then`. Tag `task-4`.

STARTER: `SPECS/UC-01-BORROW-BOOK.MD`

```
USE CASE UC-01  Borrow book
Scope         : Campus Library System (system use case)
Level         : User goal
Primary actor : Member
Stakeholders  : Member - TODO; Library - TODO; Director (offstage) - TODO
Preconditions : TODO (identity? card valid? at most how many loans? fine limit
R4?)
Minimal guar. : TODO (what is still true if step 5 fails?)
Success guar. : TODO (loan record, due date, copy status, notification?)
Trigger       : Member presents a copy and the university card at a terminal or
kiosk

Main success scenario
  1. Member presents the university card
  2. System authenticates the Member (UC-11 Authenticate, via University IdP)
  3. TODO
  ...
  n. System confirms the loan and shows the due date

Extensions
  2a. IdP rejects the card: TODO ... use case fails
  3a. TODO (R4: outstanding fine above 10.00)
  4a. TODO (R3: the copy is reserved for another member)

Special requirements: TODO (response time at the kiosk? accessibility?)
Business rules    : R3 (hold 3 days), R4 (0.50/day, block above 10.00)
Open issues       : TODO
```

EXPECTED OUTPUT (EXCERPT OF THE GHERKIN FILE)

```
Feature: UC-01 Borrow book

  Background:
    Given member "s1234" holds a valid university card
    And the IdP authenticates card "s1234"

  Scenario: Main success scenario
    Given copy "C-778" of "Clean Code" is on the shelf
    And "s1234" has no outstanding fine
    When "s1234" borrows copy "C-778"
    Then a loan for "C-778" exists with due date 28 days from today
```

```
    And copy "C-778" is marked ON_LOAN

  Scenario: 3a Outstanding fine above the limit
    Given "s1234" owes a fine of 12.50
    When "s1234" borrows copy "C-778"
    Then the loan is refused with reason FINE_LIMIT
    And copy "C-778" is still AVAILABLE

  Scenario: 4a Copy reserved for another member
    Given copy "C-778" is reserved for "s9999"
    When "s1234" borrows copy "C-778"
    Then the loan is refused with reason RESERVED
```

Show hints

- A precondition is something the system will *not* check inside the scenario. "Member is authenticated" cannot be a precondition of UC-01 if step 2 authenticates; choose one.
- Minimal guarantee for UC-01: "no loan record exists and the copy status is unchanged". Success guarantee: "loan exists, due date set, copy ON_LOAN, reservation queue updated".
- Extensions are numbered after the step where the condition is detected, not where the problem was caused. The fine limit is detected right after authentication, so it is 2b or 3a, not 1a.
- For UC-04 the supporting actor CampusPay appears as a step ("System requests payment from CampusPay") and its failure modes are the extensions, exactly like the Payment Gateway in the slides.
- Gherkin: keep `When` to one action; if you need two, you are describing two scenarios.

ACCEPTANCE CHECKLIST

- [ ] Three specifications exist with every header field filled (scope, level, primary actor, goal, stakeholders, trigger).
- [ ] Each has preconditions, a minimal and a success guarantee, a 6-10 step main scenario without UI vocabulary, and at least two numbered extensions that rejoin or fail explicitly.
- [ ] Business rules from R3 and R4 are referenced by id, and the included *Authenticate* appears as one step, not copied.
- [ ] `UC-01-borrow-book.feature` has a Background, a main scenario and one scenario per extension with the extension id in its title.

5 Traceability and review Hard 60 min

GOAL

Prove that every requirement is covered and every use case is testable, review a colleague's flawed diagram, and deliver the final clean diagram of the Campus Library System.

STEPS

1. Create `traceability.md` with a matrix: rows R1-R10, columns your use case ids, an `x` where a use case (or one of its extensions, write `x (3a)`) covers the requirement. Add a last column "test class" naming the planned JUnit 6 class per use case (`BorrowBookTest`, ...), one method name per scenario for the three specified use cases.
2. Read the matrix in both directions and write the findings under it: requirements with no use case, use cases with no requirement (gold plating?), use cases with no test. Resolve each finding: add a use case, remove one, or record an open question for the product owner.
3. Review the flawed diagram described below in `review.md`: list at least eight distinct mistakes, each with the rule it breaks (cite the slide checklist) and the concrete fix.
4. Produce `diagrams/library-final.puml`: start from `library-v2`, apply anything the matrix or the review taught you, and add the `<<time>>` actor or note for the reminder if you have not done so.
5. Run `-checkonly` and render to PNG; verify against the six-point checklist from the slides and paste the checklist with ticks into `README.md`.
6. Tag `task-5`.

THE FLAWED DIAGRAM TO REVIEW (DESCRIBED IN TEXT)

A colleague submitted `library-draft.puml`. Rendered, it shows:

- Actors: *Student*, *Staff*, *Librarian*, *Database*, *Kiosk*, *Email Server*, *Bob (head librarian)*. Student and Staff are not related; *Email Server* is drawn on the left next to Student; *Database* is drawn inside the rectangle.
- Use cases: *Login*, *Scan card*, *Search*, *Click borrow*, *Update loan table*, *Send email*, *Borrow book*, *Return book*, *Calculate fine*, *Pay fine*, *Print receipt*, *Manage catalogue*, *Backup database*, *Library*.
- Relationships: *Login* has «include» arrows pointing to every other use case; *Borrow book* ..> *Scan card* «extend»; *Return book* ..> *Pay fine* «include»; *Pay fine* ..> *Calculate fine* ..> *Update loan table* ..> *Send email* chained with «include» to show the order; a generalization triangle sits at *Student* on the line to *Staff*.
- There is no rectangle around the use cases except a dashed box labelled "Version 3"; no ids; *Search* is connected to nobody; there are 14 ellipses and 21 arrows.

STARTER: `TRACEABILITY.MD`

```
| Req | UC01 | UC02 | UC03 | UC04 | UC05 | UC06 | UC07 | UC08 | UC09 | UC10 |
UC11 | Test class (JUnit 6)         |
|-----|------|------|------|------|------|------|------|------|------|------|---
---|------------------------------|
| R1  | x    |      |      |      |      |      |      |      |      |      | x
| BorrowBookTest                |
| R2  |      | x    |      |      |      |      |      |      |      |      |
| ReturnBookTest                |
| R3  | x(4a)|      | x    |      |      |      |      |      |      |      |
| TODO                          |
| R4  | x(3a)| x(5a)|      | x    |      |      |      |      | x    |      |
| TODO                          |
| ... |      |      |      |      |      |      |      |      |      |      |
|                               |
Findings
- forward  (R -> UC): TODO e.g. "R8 fine-issued e-mail: no use case or step
covers it"
- backward (UC -> R): TODO e.g. "UC12 Request purchase has no requirement: open
question OQ-1"
- UC -> test: TODO e.g. "UC10 reminders: how to test time? fixed Clock (Ch. 06)"
Scenario methods
  BorrowBookTest: mainSuccessScenario, ext2aCardRejected, ext3aFineAboveLimit,
ext4aCopyReserved
  ReturnBookTest: TODO
  PayFineTest   : TODO
```

EXPECTED OUTPUT (EXCERPT OF `REVIEW.MD`)

```
| # | Mistake in library-draft                    | Rule broken (slide
checklist)          | Fix                                          |
|---|---------------------------------------------|-----------------------------
-----------|----------------------------------------------|
| 1 | Database and Kiosk drawn as actors          | actors are external roles
only (3)     | remove; DB is inside, kiosk is a channel     |
| 2 | Login includes every use case               | include points base ->
included (4)    | reverse: Borrow, Return ... ..> Authenticate |
| 3 | Scan card, Click borrow, Print receipt       | every use case is a goal
(2)           | steps of Borrow book scenario                |
| 4 | Pay fine -> Calculate fine -> ... chained    | no functional decomposition
(4)        | one use case; order goes to activity diagram |
| 5 | Return book includes Pay fine               | not every return has a fine
| Pay fine ..> Return book <<extend>> [fine]   |
| 6 | Triangle at Student                         | triangle points to the
parent (4)      | Student --|> Member, Staff --|> Member       |
| 7 | Bob (head librarian) as actor               | roles, not people (3)
| Librarian (Bob is an instance)              |
| 8 | No named boundary, "Version 3" box          | boundary named and drawn (1)
```

```
| rectangle "Campus Library System"            |
| 9 | Email Server on the left                    | supporting actors right (3)
| move; connect to Reserve book, Reminders     |
|10 | 14 ellipses, Search unconnected, no ids     | readable in ten seconds, ids
(5, 6)    | 10 use cases, Visitor -- Search, UC-nn ids   |
```

Show hints

- Build the matrix from the README table of Task 2 (requirements column) and then *invert* it: a column with no `x` is a use case nobody asked for.
- R8 is the usual gap: the "fine issued" e-mail is neither a goal nor covered unless UC-02's late-return extension says "System queues a fine-issued e-mail (Mail relay)". Add the step, not a use case.
- Testing time-triggered use cases: inject `java.time.Clock` and use `Clock.fixed(...)` as in Lab-06; write that in the test column.
- In the review, quote the checklist number from the slide "Checklist for a good use case diagram" for every finding; a fix must be a concrete PlantUML change, not "improve".
- The final diagram may legitimately differ from v2: the review often shows that your own draft had a chained include or a missing Visitor association.

ACCEPTANCE CHECKLIST

- [ ] The matrix covers R1-R10 against every use case id, names one test class per use case and scenario methods for UC-01, UC-02 and UC-04.
- [ ] Findings in both directions are written down and each is resolved (change, removal or open question with an id).
- [ ] `review.md` lists at least eight distinct mistakes, each with the checklist rule and a concrete PlantUML fix.
- [ ] `library-final.puml` passes `-checkonly`, has 8-12 use cases with ids, correct arrow directions, and the ticked checklist is in the README.

★

Challenge: inter-library loans, packages, scope document and test plan

Hard **90-120 min · optional, extra credit**

SCENARIO

The university joins a regional consortium: a member may request a title the campus library does not own, and the partner library ships it; in return the campus library lends its own copies to partner libraries. Partner libraries exchange requests through the consortium's **ILL Hub** API. The director now asks for a scope document to take to the consortium meeting and for a test plan the development team can start filling.

REQUIREMENTS

1. **Model the subsystem** in `diagrams/ill.puml`: the partner library (via the ILL Hub) as an external system actor that is *primary* in some use cases (it sends a lending request to us) and *supporting* in others (we ask it for a title). At least: *Request inter-library loan* (Member), *Receive partner item* and *Return partner item* (Librarian), *Lend copy to partner* (Partner library). Reuse *Authenticate* and the Mail relay; add the extensions that matter (title not available in the consortium; partner item overdue).
2. **Package diagram** `diagrams/packages.puml`: group all use cases into packages *Circulation*, *Catalogue*, *Payments*, *Notifications* and *Inter-library loans*; show «use» dependencies between packages (which package includes or extends use cases of another). One diagram per package with 3-6 use cases is the goal for a real project; here draw the package overview plus the ILL package diagram.
3. **Scope document** `docs/scope.md`, two pages: purpose and business goals, actors, in-scope use cases (id, name, primary actor, one-line goal, priority Must/Should/Could), out-of-scope list with reasons, assumptions and dependencies (IdP, CampusPay, ILL Hub), open questions with owners. Use the use cases as the backbone of every section.
4. **Test plan skeleton** in `src/test/java/edu/itc/library/usecases/`: one JUnit 6 class per use case, `@Tag("UC-nn")`, `@DisplayName` with id and name, one `@Test` method per scenario of your specifications, all bodies `fail("not implemented")` or `@Disabled`. A minimal Maven project with `spring-boot-starter-test` (JUnit 6, Mockito 5, AssertJ) so `./mvnw test -Dgroups=UC-01` runs.
5. Update `traceability.md` so that the new use cases and test classes appear, and the ILL requirements (write them as R11-R14) are covered.

SKELETON

```
@startuml packages
left to right direction
package "Circulation" {
  usecase "Borrow book"  as UC01
  usecase "Return book"  as UC02
  usecase "Reserve book" as UC03
}
package "Payments" {
  usecase "Pay fine" as UC04
}
```

```
  package "Inter-library loans" {
    usecase "Request inter-library loan" as UC13
    usecase "Lend copy to partner"       as UC16
  }
  package "Identity" {
    usecase "Authenticate" as UC11
  }
  Circulation ..> Identity : <<use>>
  Payments    ..> Circulation : <<use>>
  "Inter-library loans" ..> Circulation : <<use>>
  @enduml
```

```
  package edu.itc.library.usecases;

  import static org.junit.jupiter.api.Assertions.fail;
  import org.junit.jupiter.api.DisplayName;
  import org.junit.jupiter.api.Tag;
  import org.junit.jupiter.api.Test;

  /** Test plan for UC-13; scenarios from specs/UC-13-request-ill.md. */
  @Tag("UC-13") @Tag("R11")
  @DisplayName("UC-13 Request inter-library loan")
  class RequestInterLibraryLoanTest {

      @Test @DisplayName("main success scenario: request forwarded to ILL Hub")
      void mainSuccessScenario() { fail("not implemented"); }

      @Test @DisplayName("3a title not available in the consortium")
      void ext3aTitleNotAvailable() { fail("not implemented"); }

      @Test @DisplayName("5a ILL Hub unreachable: request queued for retry")
      void ext5aHubUnreachable() { fail("not implemented"); }
  }
```

EXPECTED OUTPUT

```
  $ java -jar tools/plantuml.jar -checkonly diagrams/*.puml        (no output: all
  valid)
  $ ./mvnw -q test -Dgroups=UC-13
  [ERROR] Tests run: 3, Failures: 3 ...
  RequestInterLibraryLoanTest.mainSuccessScenario
          not implemented                                  ← expected: the plan,
  not the code
  docs/scope.md: 2 pages, 16 use cases (UC-01..UC-16) with priority, 6 actors,
                 7 out-of-scope items, 4 open questions with owners
```

Show hints

- An actor can be primary in one use case and supporting in another; draw the Partner library once, on the right, and let the association direction in the specification (trigger field) say who starts.
- PlantUML packages: a use case may be referenced across packages by its alias; a dependency between packages is `PackageA ..> PackageB : <<use>>` (quote names with spaces).
- Scope document skeleton: 1 Purpose · 2 Actors · 3 Use cases (table) · 4 Out of scope · 5 Assumptions and dependencies · 6 Open questions. Two pages means roughly 700 words plus the table.
- Surefire runs tags with `-Dgroups=UC-13` out of the box with JUnit 6; a failing skeleton is the correct state for a test plan.
- Generate the project with `curl https://start.spring.io/starter.zip -d bootVersion=4.1.1 -d javaVersion=25 -d dependencies=` (no dependencies: the test starter comes anyway).

ACCEPTANCE CHECKLIST

- [ ] `ill.puml` models at least four ILL use cases with the Partner library as an external actor, reuses *Authenticate*, and has two justified extensions.
- [ ] `packages.puml` groups every use case into one of five packages and shows «use» dependencies without cycles.
- [ ] `docs/scope.md` is use-case driven (ids everywhere), two pages, with priorities, out-of-scope reasons and owned open questions.
- [ ] One tagged JUnit 6 class per use case with one method per specified scenario; `./mvnw test -Dgroups=UC-01` selects only that class; traceability updated.

Grading rubric

Component	Points	Full marks when…
Task 1 · Actors and boundary	10	Complete actor table with primary/supporting, rejected candidates justified, boundary and out-of-scope list, context diagram.
Task 2 · First diagram	15	8-12 goal-level use cases with ids covering R1-R10, associations only, correct sides, readable PNG, source passes `-checkonly`.
Task 3 · Include, extend, generalization	20	Authenticate extracted correctly, Pay fine extends Return book with condition, Student/Staff generalization with a real difference, justification table incl. a rejected relationship.

Component	Points	Full marks when…
Task 4 · Specifications and Gherkin	20	Three fully dressed specs with guarantees, numbered scenarios, two or more extensions each, rules referenced; Gherkin file mirrors UC-01 scenario by scenario.
Task 5 · Traceability and review	25	Two-way matrix with findings resolved, eight or more review items with rule and fix, clean final diagram with ticked checklist.
Diagram and document quality	10	Consistent ids and names across diagrams, specs, matrix and README; text sources and exports both committed; tags per task.
★ Challenge (extra credit)	+20	All four acceptance items: ILL diagram, package diagram, scope document, runnable tagged test plan, matrix updated.
Total	**100 (+20)**	

Automatic deductions: −5 per UI step or internal component (database, kiosk, service) drawn as a use case or actor (max −15); −5 per «include», «extend» or generalization arrow pointing the wrong way; −10 if the system boundary is missing or an external system is drawn inside it; −5 per use case without an id or a verb; −10 when a submitted `.puml` has no PNG/SVG export or does not pass `-checkonly`; −5 per specification with fewer than two extensions.

Submission

1. Repository layout:

```
lab-07/
├── README.md                ← name, tool, use case table (Task 2),
justification table (Task 3), ticked checklist
├── actors.md                ← Task 1
├── diagrams/
│   ├── context.puml + .png
│   ├── library-v1.puml + .png
│   ├── library-v2.puml + .png
│   ├── library-final.puml + .png
│   └── ill.puml, packages.puml (+ .png)      ← challenge
├── specs/
│   ├── UC-01-borrow-book.md, UC-02-return-book.md, UC-04-pay-fine.md
│   └── UC-01-borrow-book.feature
├── traceability.md · review.md
```

```
├── docs/scope.md                                ← challenge
└── pom.xml · src/test/java/edu/itc/library/usecases/*Test.java   ←
challenge
```

2. Verification that must pass from a clean checkout:

```
java -jar tools/plantuml.jar -checkonly diagrams/*.puml          # prints
nothing when all sources are valid
java -jar tools/plantuml.jar -Playout=smetana -tpng diagrams/*.puml   #
regenerates every PNG
./mvnw -q test -Dgroups=UC-01                                      #
challenge only: the tagged skeleton runs
```

3. Branch `lab-07`, merge request titled `Lab-07 – <your name>`, tags `task-1` … `task-5` (and `challenge`).

Back to Chapter 07 slides | Retake the chapter quiz | ← Lab-06